

ISLAMIC UNIVERSITY OF TECHNOLOGY



SWE 4301

LECTURE 20

Topic: Generics

March 26, 2025

PREPARED BY

Maliha Noushin Raida

Lecturer, Department of CSE

Contents

1	Generics	3
1.1	Generic 1: Use Generic Class	3
1.2	Generic 2 – Define a Generic <T> Class	4
1.2.1	Generic <T> Operations:	4
1.2.2	Generic <T> Method:	6
1.2.3	Generic <?> Method:	6
1.3	Generic 3 – ?/T with “extends” Generics	7
1.3.1	?/T Extends Method	8

1 GENERICS

An interface or class may be declared to take one or more type parameters, which are written in angle brackets and should be supplied when you declare a variable belonging to the interface or class or when you create a new instance of a class.

Generics means parameterized types. The idea is to allow a type (like Integer, String, etc., or user-defined types) to be a parameter to methods, classes, and interfaces. Using Generics, it is possible to create classes that work with different data types. An entity such as a class, interface, or method that operates on a parameterized type is a generic entity.

```
1 List<String> words = new ArrayList<String>();
2 words.add("Hello ");
3 words.add("world!");
4 String s = words.get(0)+words.get(1);
5 assert s.equals("Hello world!");
```

Three Uses Of Generics Here are the three common use patterns of generics

- Using a generic class, like using ArrayList<String>
- Writing generic code with a simple <T> or <?> type parameter
- Writing generic code with a <T extends Foo> type parameter

1.1 Generic 1: Use Generic Class

Many Java library classes have been made generic, so instead of just raw Object, they can be used in a way that indicates the type of object they hold.

For example, the code below creates an ArrayList that holds Strings. Both the variable holding the pointer and the call to the constructor have <String> added.

```
1 ArrayList<String> strings = new ArrayList<String>();
2 strings.add("hi");
3 strings.add("there");
4 String s = strings.get(0); // no cast required!
```

The advantage is that the compiler knows that the add() and get() methods take and return Strings, and so it can do the right type checking at compile time. We do not have to put in the (String) cast. The code that uses the list is now more readable. As a benefit of compile time typing, Eclipse code-assist now knows that strings.get(0) is a String and so can do code completion.

The type of the iterator – e.g. Iterator<String> – must match the type of the collection.

The plain type “List” without any generic type is known as the “raw” version. Raw versions still work in Java, and they essentially just contain pointers of type Object. You can assign back and forth between generic and raw versions, and it works, although it may give a warning.

In Java before generics, the same code would be written as follows;

```
1 List words = new ArrayList();
2 words.add("Hello ");
3 words.add("world!");
4 String s = ((String)words.get(0))+((String)words.get(1))
5 assert s.equals("Hello world!");
```

At runtime, all the casts are checked in any case, so if the wrong sort of object gets into a List<String> it will be noticed at runtime. This is why Java lets us mix between List<String> and raw List with just a warning– it’s all being checked at runtime anyway as a last resort.

1.2 Generic 2 – Define a Generic <T> Class

You can define your own class as a generic class. The class definition code is parameterized by a type, typically called <T>. This is more or less what ArrayList does. At the very start of the class, the parameter is added like this:

```
1 public class Foo<T> {
```

In the simplest case, the class code does not have any specific constraint on what type T is. Inside the class, T may be used in limited ways:

- declare variables, parameters, and return types of the type T
- use = on T pointers
- call methods that work on all Objects, like .equals()

1.2.1 Generic <T> Operations:

In the source code implementing the generic class, T may be used to declare variables and return types, and may be used to declare local generic variables, such as a List<T>. Essentially, T behaves like a real type such as String or Integer.

At a later stage in the compilation, T is replaced by Object. So at run-time, the notion of T is gone – it’s just Object by then.

Remember: where you see “T”, it is just replaced by “Object” to produce the code for runtime. So the ArrayList<String> code and the ArrayList<Integer> code ... those two are actually just the ArrayList<Object> code at runtime.

```
1 public class Pair<T> {
2     private T a;           // Can declare T variables
3     private T b;
4     private List<T> unused; // Can use T like this too
5
6     public Pair(T a, T b) {
7         this.a = a;
8         this.b = b;
9     }
10
11    public T getA() {
12        return a;
13    }
14
15    public T getB() {
16        return b;
17    }
18
19    public void swap() {
20        T temp = a; // NOTE T temporary variable
21        a = b;
22        b = temp;
23    }
24
25    // True if a and b are the same
26    public boolean isSame() {
27        return a.equals(b);
28        // NOTE Can only do things on T vars that work on any Object
29    }
30
31    // True if a or b is the given object
32    // NOTE: use of T as a parameter
33    public boolean contains(T elem) {
34        return (a.equals(elem) || b.equals(elem));
35    }
36
37    public static void main(String[] args) {
38        // ** Make a Pair of Integer
39        Pair<Integer> ipair = new Pair<Integer>(1, 2);
40        Integer a = ipair.getA();
41        int b = ipair.getB(); // unboxing
42    }
```

```

43 // ** Also make a Pair of String
44 Pair<String> spair = new Pair<String>("hi", "there");
45 String s = spair.getA();
46 }
47 }

```

1.2.2 Generic <T> Method:

Rather than making a whole class generic, the generic syntax can work on a single method. The syntax is that the <T> goes before the return type:

```

1 public <T> void foo(List<T> list) {

```

Here is an example where T allows the method to work for any sort of List:

```

1 // <T> Method -- use a <T> type on the method to
2 // identify what type of element is in the collection.
3 // The <T> goes just before the return type.
4 // T can be used to declare variables, return types, etc.
5 // This is ok, but slightly heavyweight, since in this case
6 // we actually don't care what type of thing is in there.
7 // This removes elements that are == to an adjacent element.
8 public static <T> void removeAdjacent(Collection<T> coll) {
9     Iterator<T> it = coll.iterator();
10    T last = null;
11    while (it.hasNext()) {
12        T curr = it.next();
13        if (curr == last)
14            it.remove();
15        last = curr;
16    }
17 }

```

1.2.3 Generic <?> Method:

The <?> generic parameter is like a <T> parameter but a little simpler. The “?” is a wildcard which means “don’t care” – a type that can be anything, since the code does not care what it is. Here is a bar() that uses “?” to take a list of anything. The syntax is a little more simple, since there is not an extra <T> required before the return type. The <?> just goes in the argument list and that’s it:

```

1 void bar(List<?> list) {

```

We can use the list inside `bar()`, but we cannot assume anything about what type it contains – we don't even have a name for the type it contains.

Inside the `List<?>` method, we can make local generic variables that mention the `<?>`, but that's the only use of `<?>` that works, such as:

```
1 List<?> temp = list;
2 Iterator<?> it = list.iterator();
```

We cannot declare a variable of type `"?"`, however, `"Object"` works fine.

```
1 Object elem = list.get(0);
```

You do not return a `<?>` type – in that case you want a `<T>` in the return type and also somewhere else specified by the client, such as a parameter.

Use Case: It is typically used when you want to accept any type, but without specifying or restricting the type. This is useful in cases where the specific type doesn't matter for the method's functionality. `<T>` is more type-safe and should be preferred when you know the type and want the benefits of generics, like avoiding casting.

`<?>` is more flexible but comes with less safety. It can accept any type but is often used when you don't need to know the exact type and just want to work with a broader set of types.

```
1 // Using <T> (preferred for type safety)
2 public static <T> void copyListWithT(List<T> source, List<T>
   destination) {
3     for (T element : source) {
4         destination.add(element);
5     }
6 }
7
8 // Using <?> (more flexible, but less type-safe)
9 public static void copyListWithWildcard(List<?> source, List<Object>
   destination) {
10    for (Object element : source) {
11        destination.add(element);
12    }
13 }
```

1.3 Generic 3 – `?/T` with “extends” Generics

The plain `<?>/<T>` generics doesn't let us assume anything about `T`. The "extends" form allows us to add a constraint about `T`. For example, with a `<T extends Number>` – for any `T` value, we

can assume it is a Number subclass, so .intValue() etc. just work. This is known as a “bounded wildcard”.

This technique can be used on a whole class, such as a Pair that contains numbers:

```
1 public class PairNumber <T extends Number> {
2     private T a;
3     private T b;
4     public PairNumber(T a, T b) {
5         this.a = a;
6         this.b = b;
7     }
8     public int sum() {
9         // Note: here we can use Number messages, like intValue().
10        return (a.intValue() + b.intValue());
11    }
12    .....
```

1.3.1 ?/T Extends Method

The “extends” constraint also works on ?/T methods,

```
1 // ?-Extends -- rather than Collection<?>, the "extends Number"
2 // adds a constraint that the elements must be subclasses of Number.
3 // The Number subclasses include Integer, Double, Long, etc.
4 // Returns the int sum of all the numbers in the collection.
5 public static int sumAll(Collection<? extends Number> nums) {
6     int sum = 0;
7     for (Number num : nums) {
8         sum += num.intValue(); // All the Number types respond to intValue().
9     }
10    return sum;
11 }
```